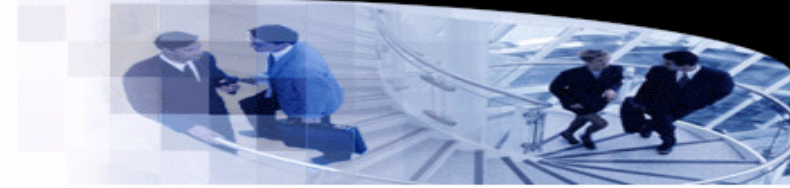




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

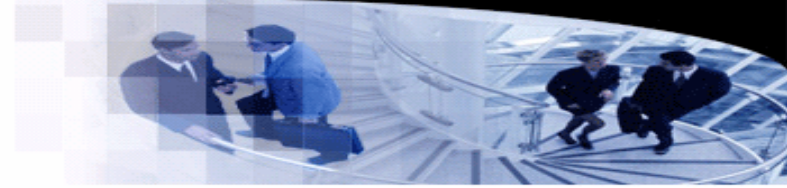
Lecture 29. Machine Code Generation

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Machine Code Generation

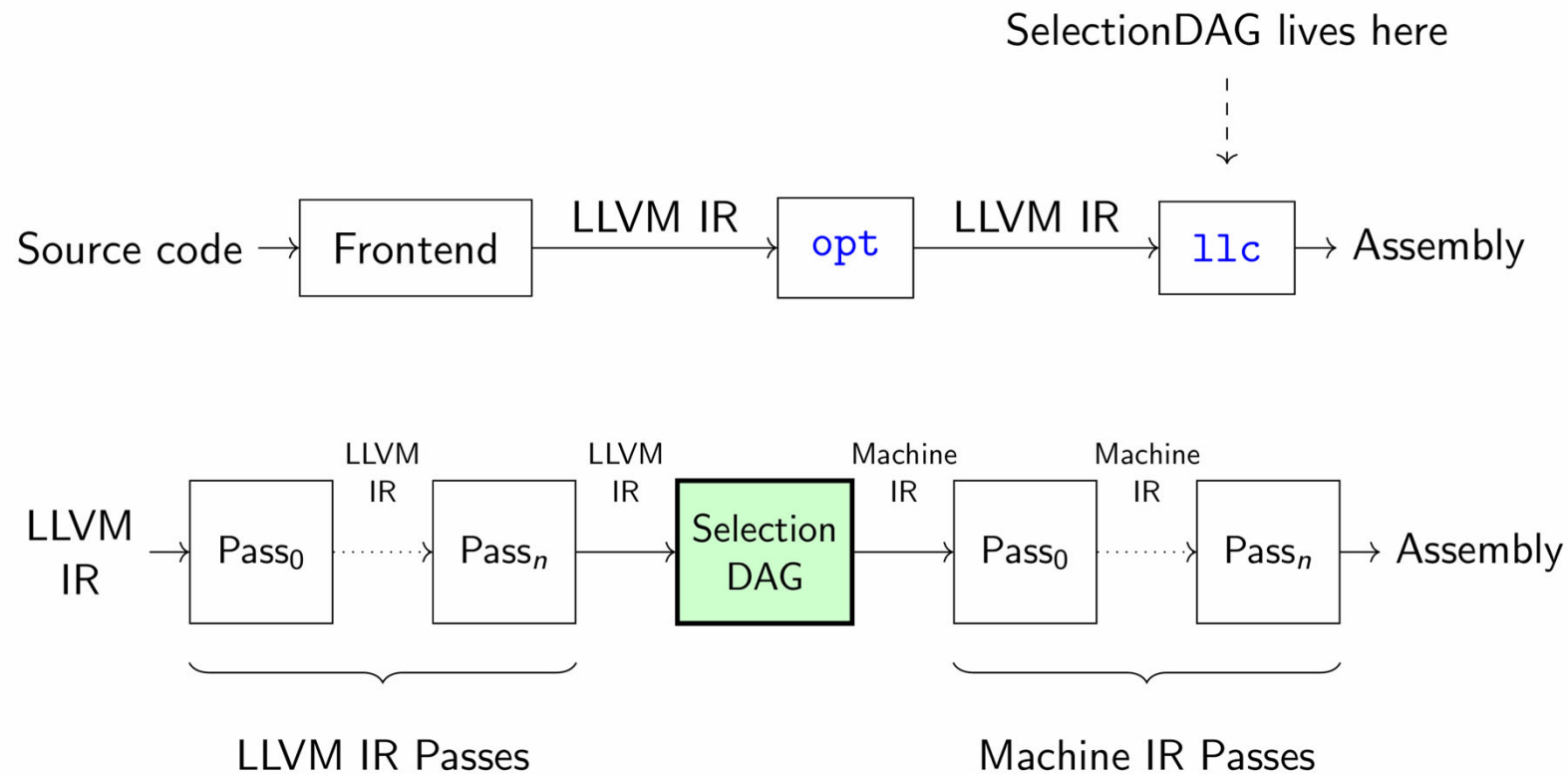


- Overview

- The compiler transforms the input source program into intermediate representations.
- High-level representations are then transformed into low-level representations.
- Once the representations are similar to assembly, the compilation is done.
- In this lecture, we demonstrate how to generate machine code via LLVM.
- A beginner's guide to SelectionDAG
 - <https://llvm.org/devmtg/2024-10/slides/tutorial/MacLean-Fargnoli-ABeginnersGuide-to-SelectionDAG.pdf>

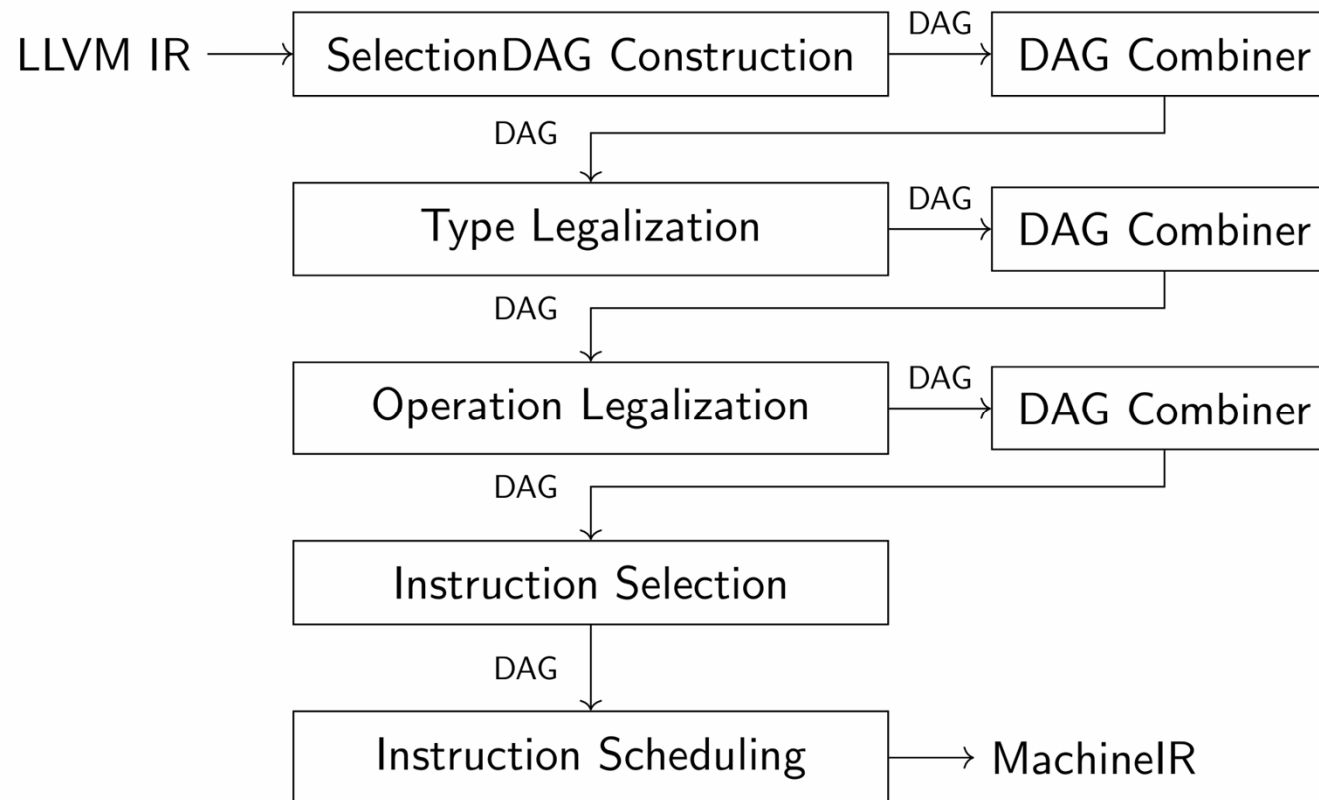
Machine Code Generation

- Compilation flow of LLVM



Machine Code Generation

- SelectionDAG code generation flow



Machine Code Generation

- SelectionDAG construction

- The program is represented as a CFG.
- Each basic block is a SelectionDAG.

```
int a, b, c;
```

```
void foo() {  
    if (a == 0)  
        b = 10;  
    else  
        c = 20;  
}
```

```
define dso_local void @_Z3foov() #0 {  
entry:  
    %0 = load i32, ptr @a, align 4  
    %cmp = icmp eq i32 %0, 0  
    br i1 %cmp, label %if.then, label %if.else
```

```
if.then:  
    store i32 10, ptr @b, align 4  
    br label %if.end
```

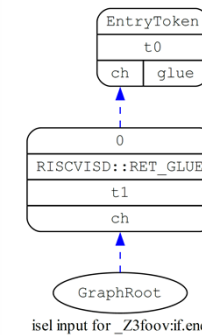
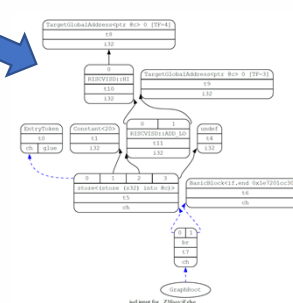
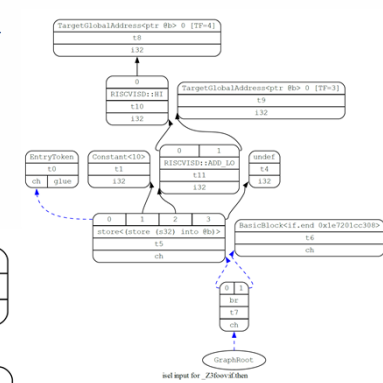
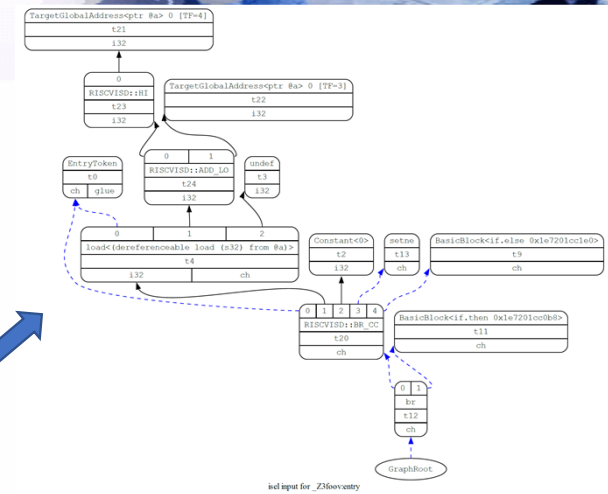
```
if.else:  
    store i32 20, ptr @c, align 4  
    br label %if.end
```

```
if.end:  
    ret void  
}
```

; preds = %entry

; preds = %entry

; preds = %if.else, %if.then

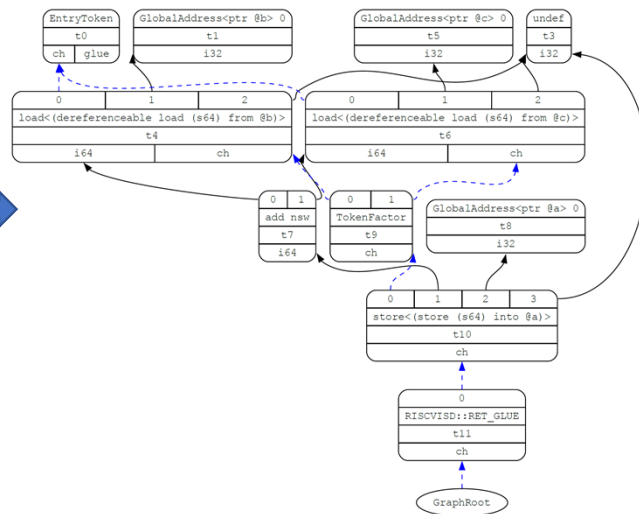


Machine Code Generation

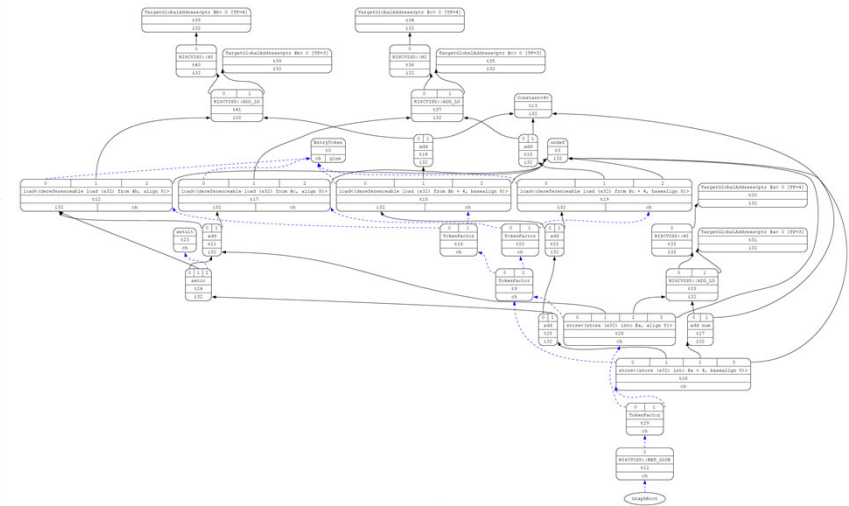
- Type legalization

- The higher level IR always contains richer types than that of lower level IR.
- Convert illegal types into legal types in the target machine.

```
long long a, b, c;  
  
void foo() {  
    a = b + c;  
}
```



legalize-types input for _Z3fooventry



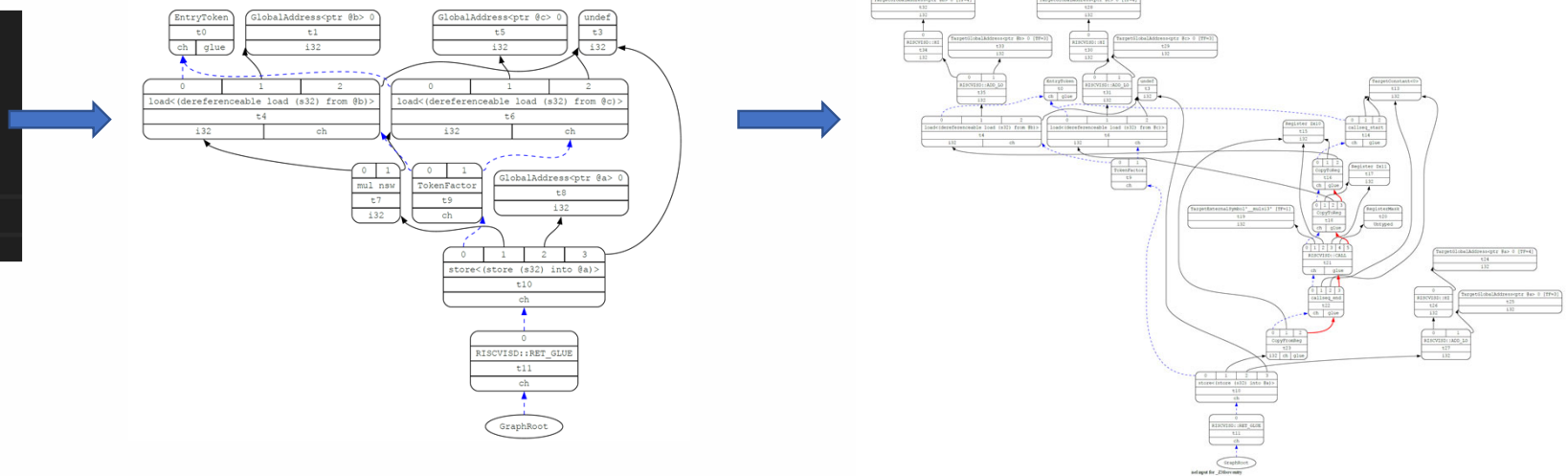
output for _Z3fooventry

Machine Code Generation

- Operation legalization

- Translate illegal operation into legal ones in the target machine.
- E.g., multiplication is not supported in rv32i.

```
int a, b, c;  
  
void foo() {  
    a = b * c;  
}
```

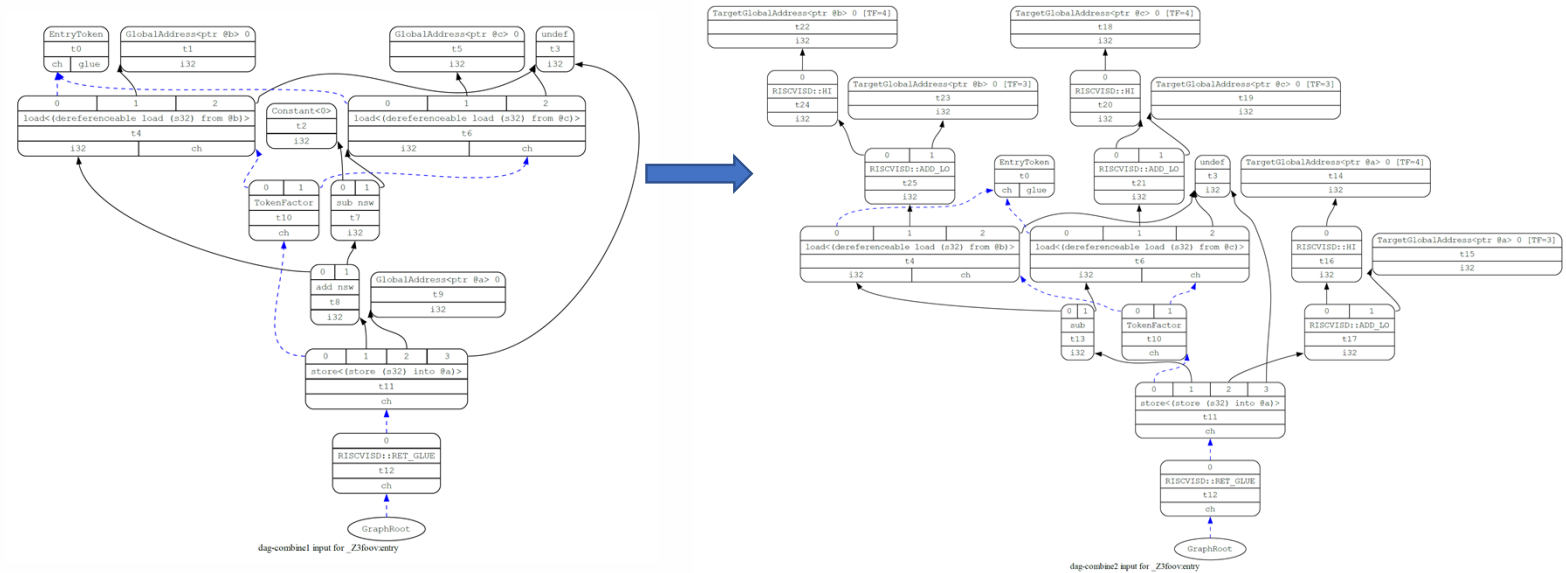


Machine Code Generation

• DAG Combiners

- Perform peephole optimizations on SelectionDAG.
- For example, replace $b + (-c)$ with $b - c$.

```
int a, b, c;  
  
void foo() {  
    a = b + (-c);  
}
```



Machine Code Generation

- **Instruction selection**

- A tree-pattern-matching approach.
- Replace a tree with a machine instruction node(s).
- Traverse the DAG in topological order such that operator always selected before its operands.
- Pattern matching heuristics:
 - Prefer more complex patterns.
 - Prefer lower emitted instruction count.
 - Prefer larger match pattern size.

